

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе № 2

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

Реализация микросервисов сервиса «Lecter's Recipes»

Вариант 5

Выполнил: студент группы БР1.1

Кошкарев Кирилл Павлович

Проверил:

Санкт-Петербург, 2026

1. Цель работы

Опираясь на документ ДЗ4, реализовать разделение бэкенд-приложения на микросервисы.

2. Реализованный контур

Создано четыре независимых Node.js-приложения. Каждое имеет собственный `package.json`, собственную конфигурацию и собственную базу данных; общего кода на уровне пакетов нет — небольшой набор служебных модулей (ошибки, пагинация, проверка JWT, `middleware`) скопирован в каждый сервис, чтобы сервисы можно было собирать и развёртывать независимо.

```
labs/lab2/
├── api-gateway/      :3000  маршрутизация, агрегация, Swagger, витрина
├── user-service/     :3001  учётные записи, JWT, подписки      → lecters_users
├── recipe-service/   :3002  рецепты, справочники, поиск        → lecters_recipes
├── social-service/   :3003  лайки, избранное, комментарии    → lecters_social
└── integration-smoke.mjs  сквозной сценарий проверки контура

Структура каждого сервиса:
src/
├── models/          сущности своей базы
├── services/         бизнес-логика
├── controllers/      публичные и внутренние обработчики
├── routes/           /api/v1/* и /internal/*
├── messaging/        шина событий: bus.ts, publisher.ts, consumer.ts
└── middlewares/      аутентификация, валидация, обработка ошибок
```

Листинг 1 — структура микросервисного контура

3. Разделение данных

Связи между сущностями разных сервисов заменены числовыми полями. Например, в `recipe-service` поле `authorId` объявлено обычным столбцом с индексом, а не отношением:

```
/**
 * Рецепт. authorId — «мягкая» ссылка на пользователя из другой базы:
 * внешнего ключа нет, целостность поддерживается событиями и внутренним API.
 */
@Entity('recipes')
export class Recipe {
  @PrimaryGeneratedColumn({ type: 'bigint' }) id: string;
  @Index() @Column({ type: 'bigint' }) authorId: string;
  ...
}
```

Листинг 2 — ссылка на сущность чужого сервиса

В social-service добавлены таблицы recipe_projection и user_projection, а также processed_events — журнал обработанных событий для идемпотентности.

4. Единая аутентификация

Токены выпускает только user-service, но проверить их может каждый сервис: секрет JWT общий и передаётся переменной окружения JWT_SECRET. Благодаря этому шлюз не обращается к user-service ради проверки подписи, а сервисы одинаково понимают поля sub, username и role. Интеграционный сценарий отдельно проверяет, что токен, выданный при регистрации через шлюз, принимается social-service напрямую.

5. Агрегация ответов в шлюзе

Карточка рецепта в публичном API содержит автора и состояние реакций — данные трёх сервисов. Шлюз запрашивает у recipe-service список карточек, затем одним пакетным вызовом получает профили авторов и одним — счётчики с признаками liked/favorite, после чего сшивает результат. Число межсервисных вызовов не зависит от размера страницы: три вызова вместо $2N + 1$.

```
export const enrichRecipes = async (cards, viewerId) => {
  const [users, reactions] = await Promise.all([
    fetchUsers(cards.map((card) => card.authorId)), // /internal/users?ids=
    fetchReactions(cards.map((card) => card.id), viewerId), // /internal/social/recipes?ids=
  ]);
  return cards.map((card) => ({
    ...card,
    author: users.get(String(card.authorId)) ?? fallbackAuthor(card.authorId),
    likesCount: reactions.get(String(card.id))?.likesCount ?? card.likesCount ?? 0,
    isLiked: reactions.get(String(card.id))?.liked ?? false,
    isFavorite: reactions.get(String(card.id))?.favorite ?? false,
  }));
};
```

Листинг 3 — сшивание данных трёх сервисов в шлюзе

6. Устойчивость к отказам

Вызовы разделены на обязательные и второстепенные. Обязательный вызов (например, получение списка рецептов) при недоступности сервиса возвращает клиенту 503 или 504. Второстепенный (профили авторов, счётчики реакций) выполняется «мягко»: при ошибке в журнал пишется предупреждение, а карточки отдаются с запасными значениями — витрина

продолжает работать, теряя лишь имена авторов и счётчики.

```
export const internalGetSoft = async (url, fallback) => {
  try {
    return await internalGet(url);
  } catch (error) {
    console.warn(`[gateway] деградация: ${url} → ${error.message}`);
    return fallback;
  }
};
```

Листинг 4 — деградация вместо отказа всего ответа

Аналогично ведёт себя `social-service` при отсутствии рецепта в проекции: он синхронно запрашивает сводку у `recipe-service` и прогревает проекцию, поэтому лайк не теряется даже при отставании событий. Именно этот путь работает, если брокер сообщений не поднят: контур остаётся работоспособным.

7. Проверка контура

Написан сценарий `integration-smoke.mjs` из 38 проверок. Он поднимается на уровень публичного API и дополнительно стучится во внутренние эндпоинты, проверяя их защиту.

```
$ node integration-smoke.mjs
INTEGRATION: микросервисы Lecter's Recipes @ http://localhost:3000

1. Здоровье контура           ☐ gateway опрашивает все сервисы
2. Аутентификация            ☐ POST /auth/register через gateway → 201
3. Единый токен               ☐ тот же токен принят social-service напрямую
4. Витрина                   ☐ gateway подставил автора из user-service
                              ☐ gateway подставил состояние реакций из social-service
5. Фильтрация                 ☐ фильтр difficulty+maxCookTime работает через gateway
6. Социальные действия       ☐ GET /users/me/favorites собирает карточки из recipe-service
7. Подписки и лента           ☐ лента собрана из публикаций автора
8. Публикация рецепта         ☐ POST /recipes/:id/publish → published
9. Внутренние эндпоинты       ☐ internal без X-Internal-Token → 401
                              ☐ gateway не публикует /internal наружу
10. Изоляция баз              ☐ лайк несуществующего рецепта → 404
11. Права доступа             ☐ изменение чужого рецепта → 403

ИТОГО: 38 успешно, 0 провалено
```

Листинг 5 — результат интеграционного прогона

Дополнительно коллекция Postman из ДЗЗ запущена против шлюза без единой правки: 28 запросов и 75 проверок проходят успешно. Это подтверждает, что разделение на сервисы не изменило публичный контракт API.

8. Выводы

Монолит разделён на три сервиса и шлюз в соответствии с проектом Д34. Каждый сервис владеет своей базой, межбазовых внешних ключей нет, публичный контракт сохранён. Реализованы пакетные внутренние вызовы и деградация, поэтому отказ второстепенного сервиса не обрушивает витрину. Работоспособность подтверждена 38 проверками интеграционного сценария и повторным прогоном коллекции Д33 против шлюза.